

# KT: RCK Substack Scraper — Setup & How It Works

## 1. What This System Does

This is a **daily automated intelligence feed** that scrapes financial content from two sources and publishes it to a GitHub Pages dashboard accessible to the whole RCK Analytics team — no login required.

**Substack scraper** — runs once a day at 8:45 AM IST on the local Windows laptop. It hits ~380 Substack newsletters via RSS (with a BeautifulSoup HTML fallback when RSS is blocked) and appends any new articles to `data/articles.json`. It then commits and pushes to GitHub, which triggers GitHub Pages to refresh the live dashboard within ~2 minutes.

**Reddit scraper** — runs every 3 hours entirely on GitHub’s servers (GitHub Actions). It fetches the Reddit/stocks RSS feed, deduplicates by article link, and commits `data/reddit_articles.json` only when something actually changed.

**Dashboard ( `index.html` )** — a single HTML file served via GitHub Pages. It has two tabs (Substack and Reddit) with date filters, search, source dropdown, NEW badges for articles under 24 hours old, and visited-link dimming that persists across sessions via `localStorage`.

The core design principle: **article link = unique ID**. Nothing is ever overwritten or deleted. Data only ever grows. Every run appends; it never rebuilds from scratch.

## 2. Repo & File Structure

| File / Folder                                  | What it does                                                         |
|------------------------------------------------|----------------------------------------------------------------------|
| <code>index.html</code>                        | The dashboard — served via GitHub Pages, both Substack + Reddit tabs |
| <code>theme.css</code> / <code>theme.js</code> | Dark/light mode toggle, skeleton loader animation                    |
| <code>favicon.jpg</code>                       | RCK Analytics logo                                                   |
| <code>run.bat</code>                           | Windows batch file: pulls from Git → runs scraper → commits + pushes |

| File / Folder                                        | What it does                                                       |
|------------------------------------------------------|--------------------------------------------------------------------|
| <code>source_links.xlsx</code>                       | 380 Substack URLs (Name + URL, in the <code>archives</code> sheet) |
| <code>scraper/scrape.py</code>                       | Substack scraper — RSS + BeautifulSoup fallback                    |
| <code>scraper/scrape_reddit.py</code>                | Reddit scraper — RSS only, runs on GitHub Actions                  |
| <code>scraper/requirements.txt</code>                | Python dependencies                                                |
| <code>data/articles.json</code>                      | Substack data — grows daily, never overwritten                     |
| <code>data/reddit_articles.json</code>               | Reddit data — updated every 3 hours via GitHub Actions             |
| <code>backups/minsider_backup_YYYY-MM-DD.xlsx</code> | Daily Excel snapshots of <code>articles.json</code>                |
| <code>.github/workflows/scrape-reddit.yml</code>     | GitHub Actions cron (every 3h) for Reddit scraper                  |

The `data/` folder is the live database. The `backups/` folder is a dated audit trail — one Excel file per day the local scraper ran.

### 3. How the Scrapers Work

Substack scraper ( `scrape.py` ) — runs daily at 8:45 AM IST on the local machine

1. Loads `source_links.xlsx` (the `archives` sheet) — this is the list of ~380 Substack URLs.
2. Loads `data/articles.json` and builds a `seen_links` set — every link already in the file.
3. For each Substack, constructs the RSS feed URL ( `<base>/feed` ) and tries to fetch it.
4. **If RSS works:** parses entries, skips any link already in `seen_links` , appends the rest.
5. **If RSS is blocked or empty:** falls back to `requests` + BeautifulSoup on the archive URL, scraping `<a data-testid="post-preview-title">` elements.
6. After all sources: merges new articles into the existing JSON, deduplicates by `link` , sorts newest-first, saves `data/articles.json` , and writes a dated Excel backup to `backups/` .

| Scenario              | What happens                                           |
|-----------------------|--------------------------------------------------------|
| RSS works             | Parse feed, skip known links                           |
| RSS blocked/empty     | Fallback to BeautifulSoup on archive page              |
| Link already in JSON  | Skipped — link is the unique ID                        |
| No new articles found | <code>articles.json</code> unchanged, no commit needed |

Reddit scraper ( `scrape_reddit.py` ) — runs every 3 hours via GitHub Actions

1. Fetches two Reddit RSS URLs (r/stocks and r/investing or similar — check the `RSS_URLS` list in the script).
  2. Deduplicates against `data/reddit_articles.json` by link.
  3. Extracts the subreddit name from the link (e.g. `r/stocks` ) and stores it as the `substack` field.
  4. Commits and pushes **only if the file actually changed** — no noise in git history.
- 

## 4. Windows Task Scheduler Setup (Step-by-Step)

This is the most important thing to get right on a new machine. The task runs `run.bat` daily at 8:45 AM IST. Follow these steps exactly.

### Step 1 — Open Task Scheduler

Press `Win + S`, search **Task Scheduler**, open it.

### Step 2 — Create a new task (not a basic task)

In the right panel, click **Create Task** (not "Create Basic Task" — the basic wizard has fewer options).

### Step 3 — General tab

| Setting                              | Value                                          |
|--------------------------------------|------------------------------------------------|
| Name                                 | <code>RCK Substack Scraper</code>              |
| Run whether user is logged on or not | <input checked="" type="checkbox"/> Check this |
| Run with highest privileges          | Leave unchecked (least privilege is fine)      |

## Step 4 — Triggers tab

Click **New**.

| Setting        | Value                                       |
|----------------|---------------------------------------------|
| Begin the task | On a schedule                               |
| Settings       | Daily                                       |
| Start          | Set today's date, time = <b>8:45:00 AM</b>  |
| Recur every    | 1 days                                      |
| Enabled        | <input checked="" type="checkbox"/> Checked |

Click OK.

## Step 5 — Actions tab

Click **New**.

| Setting        | Value                                                                       |
|----------------|-----------------------------------------------------------------------------|
| Action         | Start a program                                                             |
| Program/script | "D:\Insider Monkey\run.bat" ( <i>update this path for the new machine</i> ) |
| Add arguments  | ( <i>leave blank</i> )                                                      |
| Start in       | ( <i>leave blank</i> )                                                      |

Click OK.

## Step 6 — Settings tab

These match the exported XML exactly:

| Setting                                                        | Value                                                                                                 |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Allow task to be run on demand                                 | <input checked="" type="checkbox"/>                                                                   |
| Run task as soon as possible after a scheduled start is missed | <input checked="" type="checkbox"/> ( <i>critical — catches up if the laptop was off at 8:45 AM</i> ) |
| If the task fails, restart every                               | Leave defaults                                                                                        |

| Setting                                                           | Value                                            |
|-------------------------------------------------------------------|--------------------------------------------------|
| Stop the task if it runs longer than                              | 72 hours                                         |
| If the running task does not end when requested, force it to stop | <input checked="" type="checkbox"/>              |
| If the task is not scheduled to run again, delete it              | <input checked="" type="checkbox"/> Do NOT check |
| Multiple instances                                                | Stop the existing instance                       |

## Step 7 — Conditions tab

| Setting                                            | Value                                               |
|----------------------------------------------------|-----------------------------------------------------|
| Start the task only if the computer is on AC power | <input checked="" type="checkbox"/> Uncheck         |
| Stop if the computer switches to battery power     | <input checked="" type="checkbox"/> Uncheck         |
| Wake the computer to run this task                 | <input checked="" type="checkbox"/> Leave unchecked |

Unchecking the battery conditions is important — otherwise the task silently skips if the laptop is on battery at 8:45 AM.

## Step 8 — Save and test

Click **OK**. Windows will ask for your account password — enter it.

To test immediately: right-click the task in Task Scheduler → **Run**. Watch a `cmd` window appear briefly. After ~5 minutes, check `data/articles.json` to confirm it updated.

## Importing the existing XML instead

If you have the exported XML file ( `RCK Substack Scraper.xml` ), you can import it directly:

1. Task Scheduler → **Action menu** → **Import Task**
2. Select the XML file
3. Go to **Actions tab**, update the path in the `Program/script` field to match the new machine's directory
4. Save and enter your password

⚠ After importing, always verify the path in the Actions tab — the original path `D:\Insider Monkey\run.bat` will almost certainly be wrong on a new machine.

## 5. run.bat — What It Does

`run.bat` is the glue script that orchestrates everything for the Substack side. Task Scheduler calls it; it takes care of the rest.

### Step-by-step what happens when it runs:

1. **Prints a header** — timestamp so you can see exactly when it started in logs.
2. `git pull --rebase --autostash origin main` — picks up any changes GitHub Actions pushed (e.g. a fresh `reddit_articles.json`) before the scraper starts. This prevents a merge conflict later.
3. **Runs `scrape.py`** — the actual scraper. If it exits with an error code, `run.bat` stops and shows an error.
4. `git add data/articles.json` — stages only the data file (not backups, which can be large).
5. `git diff --staged --quiet` — checks if `articles.json` actually changed. If nothing is new, it skips the commit entirely — no noise in git history.
6. `git commit -m "chore: daily scrape <date>"` — commits the new data.
7. `git pull --rebase origin main` again — re-syncs before pushing, in case GitHub Actions ran while the scraper was working.
8. `git push origin main` — pushes to GitHub, triggering GitHub Pages to update within ~2 minutes.

### Three paths to configure at the top of `run.bat` for a new machine:

```
set REPO_DIR=D:\Insider Monkey\insider-monkey-substacks
set VENV_PYTHON=D:\Insider Monkey\venv\Scripts\python.exe
set SCRIPT=D:\Insider Monkey\insider-monkey-substacks\scraper\scrape.py
```

Change all three to match wherever the repo is cloned and wherever the Python virtual environment lives on the new machine. Nothing else in the file needs to change.

## 6. Python Environment Setup

The scraper requires Python 3.11+ and a virtual environment. Do this once on any new machine.

## Clone the repo

```
git clone https://github.com/<org>/insider-monkey-substacks.git "D:\Insider Monkey\insider-monkey-substacks"
```

*(Replace the GitHub URL with the actual repo URL.)*

## Create a virtual environment

```
python -m venv "D:\Insider Monkey\venv"
```

Put the venv **outside** the repo folder — that way it is never accidentally committed.

## Install dependencies

```
"D:\Insider Monkey\venv\Scripts\python.exe" -m pip install -r "D:\Insider Monkey\insider-monkey-substacks\scraper\requirements.txt"
```

### Dependencies installed:

| Package                     | Used for                                                            |
|-----------------------------|---------------------------------------------------------------------|
| <code>requests</code>       | HTTP calls to RSS feeds and Substack archive pages                  |
| <code>beautifulsoup4</code> | HTML parsing for fallback scraping                                  |
| <code>lxml</code>           | Fast XML/HTML parser (BeautifulSoup backend)                        |
| <code>feedparser</code>     | RSS feed parsing                                                    |
| <code>pandas</code>         | DataFrame operations, deduplication, sorting                        |
| <code>openpyxl</code>       | Reading <code>source_links.xlsx</code> , writing backup Excel files |

## Verify the setup

Run the scraper once manually to confirm everything works:

```
"D:\Insider Monkey\venv\Scripts\python.exe" "D:\Insider Monkey\insider-monkey-substacks\scraper\scrape.py"
```

It should print INFO logs per source and end with a `Done. X total articles. Y new today.` line.

---

## 7. GitHub Actions — Reddit Scraper

The Reddit scraper needs **no local machine**. It runs entirely on GitHub's servers on a cron schedule. Nothing to set up beyond making sure the workflow file exists in the repo.

**Workflow file:** `.github/workflows/scrape-reddit.yml`

| Property       | Value                                                                     |
|----------------|---------------------------------------------------------------------------|
| Schedule       | Every 3 hours ( <code>0 */3 * * * UTC</code> )                            |
| Manual trigger | Available via Actions tab → <code>scrape-reddit.yml</code> → Run workflow |
| Smart commits  | Only commits if <code>reddit_articles.json</code> actually changed        |
| Runs on        | <code>ubuntu-latest</code> (GitHub-hosted runner)                         |

### What the workflow does:

1. Checks out the repo
2. Sets up Python
3. Installs requirements from `scraper/requirements.txt`
4. Runs `scraper/scrape_reddit.py`
5. Checks if `data/reddit_articles.json` changed — if yes, commits and pushes with message `chore: reddit scrape`

**GitHub Pages** is enabled on the `main` branch. Any push to `main` (whether from the local machine or GitHub Actions) triggers a Pages rebuild within ~2 minutes. No manual deploy step.

**To check if it's running:** Go to the repo on GitHub → **Actions** tab → look for `scrape-reddit` workflow runs. Green = success. If it has been red for more than 6 hours, something is broken (likely a Reddit RSS URL change).

**i** The Substack scraper also has a workflow file ( `scrape-main.yml` ) in the repo, but it is **disabled** — the Substack scraper runs locally via Task Scheduler, not on GitHub Actions.

## 8. Things to Keep in Mind

### The laptop must be on at 8:45 AM IST

The Substack scraper runs on the local machine. If the laptop is off or asleep, the scraper won't run. The Task Scheduler setting **"Run task as soon as possible after a scheduled start is missed"** (`StartWhenAvailable = true` in the XML) handles this: the moment the machine boots and connects to the internet, the task runs automatically. So a late start is fine — it catches up. A day entirely without the machine being on = that day's Substack articles arrive the next morning instead.

### After cloning to a new machine, update three things

1. The three paths in `run.bat` (`REPO_DIR`, `VENV_PYTHON`, `SCRIPT`)
2. The path in the Task Scheduler action
3. Ensure Git is configured with push access (`git config --global user.name/email` and SSH key or credential manager set up)

### If the scraper stops updating — checklist

| Symptom                                           | Likely cause                                             | Fix                                                                                   |
|---------------------------------------------------|----------------------------------------------------------|---------------------------------------------------------------------------------------|
| No new Substack articles for 1+ days              | Task Scheduler task disabled, or laptop was off all day  | Check Task Scheduler → History tab; manually run the task                             |
| <code>articles.json</code> unchanged but task ran | 0 new articles is normal — all sources were already seen | No action needed                                                                      |
| <code>git push</code> failing                     | Git credentials expired or SSH key missing               | Re-authenticate; check <code>git push origin main</code> manually in CMD              |
| BeautifulSoup fallback returning 0 results        | Substack changed its HTML structure                      | Inspect the archive page, update the CSS selector in <code>scrape.py</code>           |
| Reddit feed not updating                          | Reddit changed the RSS URL                               | Check <code>RSS_URLS</code> in <code>scrape_reddit.py</code> , test them in a browser |
| GitHub Actions all red                            | Workflow failing on GitHub's side                        | Check Actions tab, read the error log                                                 |

## `source_links.xlsx` — adding new Substacks

Open `source_links.xlsx` → `archives` sheet → add a new row with `Name` and the Substack archive URL (e.g. `https://example.substack.com/archive`). The scraper reads this file on every run — no code change needed.

## Backups are local only

The `backups/` folder (daily `.xlsx` files) is in the repo but is git-ignored via the workflow (only `data/articles.json` is committed). The Excel backups exist only on the machine that ran the scraper. If you need them, they're in the `backups/` subfolder of the repo directory.

## The dashboard is public

GitHub Pages serves `index.html` publicly (no auth). Anyone with the link can see it. The visited-link tracking (which articles you've read) is stored in your own browser's `localStorage` — it does not sync across teammates' machines.